```python
# simulate_bracket_engine.py
from Bracket.simulate_game_engine import simulate_game, Team, get_models_and_data
import os
from collections import defaultdict

score_by_round = defaultdict(list)

def simulate_tournament(noise_std=1.0, consistency_multiplier=1.0, blend_weight=0.0, normalize_by_round=False):
    db_path = os.path.join(os.path.dirname(__file__), "data", "bracket.db")
    df, model, EXPECTED_FEATURES = get_models_and_data(db_path, [])
    log_lines = []
    def log(msg=""):
        print(msg)
        log_lines.append(msg)

    def play_round(label, matchups, current_round, round_label):
        log(f"\n{label}")
        log("-" * 40)
        winners = []
        for i, (t1, t2) in enumerate(matchups, 1):
            result = simulate_game(
                t1, t1.seed, t2, t2.seed,
                db_path=db_path,
                model=model,
                expected_features=EXPECTED_FEATURES,
                noise_std=noise_std,
                consistency_multiplier=consistency_multiplier,
                blend_weight=blend_weight,
                normalize_by_round=normalize_by_round,
                current_round=current_round,
            )
            winner = result["winner"]
            s1, s2 = result["t1_score"], result["
```

```
t2_score"]
            log(f"Game {i:2d}:")
            log(f"{t1.name} (#{t1.seed}) {s1:.1f} vs {t2.name} (#{t2.seed}) {s2:.1f} → Winner: {winner.name}")
            winners.append(winner)
        return winners

    log("=" * 60)
    log("□ MARCH MADNESS TOURNAMENT SIMULATION □")
    log(f"Chaos Level (noise_std): {noise_std}")
    log(f"Seed Consistency Bias: {consistency_multiplier}")
    log(f"Model vs History Blend: {blend_weight}")
    log(f"Normalize by Round: {normalize_by_round}")
    log("=" * 60)

    # --- FIRST FOUR ---
    log("\n□ FIRST FOUR GAMES (Play-In)")
    ff_pairs = [
        (Team("Saint Francis", 16), Team("Alabama State", 16)),
        (Team("North Carolina", 11), Team("San Diego State", 11)),
        (Team("Mount St. Mary's", 16), Team("American", 16)),
        (Team("Xavier", 11), Team("Texas", 11))
    ]
    first_four_winners = play_round("First Four", ff_pairs, 64, "R64")
    ff1, ff2, ff3, ff4 = first_four_winners

    # --- ROUND OF 64 ---
    round64_pairs = [
        # SOUTH
        (Team("Auburn", 1), ff1), (Team("Louisville", 8), Team("Creighton", 9)),
        (Team("Michigan", 5), Team("UC San Diego", 12)), (Team("Texas A&M", 4), Team("Yale", 13)),
        (Team("Mississippi", 6), ff2), (Team("Iowa State", 3), Team("Lipscomb", 14)),
```

```
        (Team("Marquette", 7), Team("New Mexico", 10)), (Team("Michigan State", 2), Team("Bryant", 15)),
        # WEST
        (Team("Florida", 1), Team("Norfolk State", 16)), (Team("Connecticut", 8), Team("Oklahoma", 9)),
        (Team("Memphis", 5), Team("Colorado State", 12)), (Team("Maryland", 4), Team("Grand Canyon", 13)),
        (Team("Missouri", 6), Team("Drake", 11)), (Team("Texas Tech", 3), Team("UNC Wilmington", 14)),
        (Team("Kansas", 7), Team("Arkansas", 10)), (Team("St. John's", 2), Team("Nebraska Omaha", 15)),
        # EAST
        (Team("Duke", 1), ff3), (Team("Mississippi State", 8), Team("Baylor", 9)),
        (Team("Oregon", 5), Team("Liberty", 12)), (Team("Arizona", 4), Team("Akron", 13)),
        (Team("BYU", 6), Team("VCU", 11)), (Team("Wisconsin", 3), Team("Montana", 14)),
        (Team("Saint Mary's", 7), Team("Vanderbilt", 10)), (Team("Alabama", 2), Team("Robert Morris", 15)),
        # MIDWEST
        (Team("Houston", 1), Team("SIU Edwardsville", 16)), (Team("Gonzaga", 8), Team("Georgia", 9)),
        (Team("Clemson", 5), Team("McNeese State", 12)), (Team("Purdue", 4), Team("High Point", 13)),
        (Team("Illinois", 6), ff4), (Team("Kentucky", 3), Team("Troy", 14)),
        (Team("UCLA", 7), Team("Utah State", 10)), (Team("Tennessee", 2), Team("Wofford", 15)),
    ]
    r64_winners = play_round("▯ ROUND OF 64", round64_pairs, 64, "R64")

    # --- NEXT ROUNDS ---
    r32_winners = play_round("▯ ROUND OF 32", list(zip(r64_winners[0::2], r64_winners[1::2])), 32, "R32")
    r16_winners = play_round("▯ SWEET 16", list(zip(r32_winners[0::2], r32_winners[1::2])), 16, "R16")
```

```
    r8_winners  = play_round("⚡ ELITE 8", list(zip(r16_winners[0::2], r16_winners[1::2])), 8, "R8")
    r4_winners  = play_round("⚡ FINAL FOUR", list(zip(r8_winners[0::2], r8_winners[1::2])), 4, "R4")

    # --- CHAMPIONSHIP ---
    log("\n⚡ NATIONAL CHAMPIONSHIP")
    log("=" * 40)
    result = simulate_game(
        r4_winners[0], r4_winners[0].seed,
        r4_winners[1], r4_winners[1].seed,
        db_path=db_path,
        model=model,
        expected_features=EXPECTED_FEATURES,
        noise_std=noise_std,
        consistency_multiplier=consistency_multiplier,
        blend_weight=blend_weight,
        normalize_by_round=normalize_by_round,
        current_round=2,
    )
    champ = result["winner"]
    s1, s2 = result["t1_score"], result["t2_score"]
    log(f"{r4_winners[0].name} (#{r4_winners[0].seed}) {s1:.1f} vs {r4_winners[1].name} (#{r4_winners[1].seed}) {s2:.1f} → Winner: {champ.name}")
    log(f"\n⚡ CHAMPIONS: {champ.name} (#{champ.seed})! ⚡")

    print(f"Tournament complete! Champion: {champ.name} (#{champ.seed})")

    return {
        "champion_name": champ.name,
        "champion_seed": champ.seed,
        "log": "\n".join(log_lines)
    }
```